CSES Analysis Notebook

Curated, non-overlapping, non-trivial techniques · 31 tasks in 5 sections

Curated selection; short introductory analyses are omitted.

Contents

Data Structures

3314 · Mountain Range
3304 · Visible Buildings Queries
2184 · Missing Coin Sum Queries
1077 · Sliding Window Cost

String

3359 · Minimal Grid Path
2186 · Special Substrings
1743 · String Reorder
1112 · Required Substring
1087 · Shortest Subsequence

Graph

2177 · Strongly Connected Edges
2143 · Reachability Queries
2138 · Reachable Nodes
2080 · Fixed-Length Paths I
1686 · Coin Collector
1195 · Flight Discount
1136 · Counting Paths

Math

3397 · Permutation Order
3355 · Sum of Four Squares
2214 · Inverse Inversions
2210 · Counting Grids
2209 · Counting Necklaces
2182 · Divisor Analysis
2164 · Josephus Queries
2137 · Corner Subgrid Count
1728 · Inversion Probability
1726 · Moving Robots
1725 · Dice Probability
1082 · Sum of Divisors

Dynamic Programming

2229 · Permutation Inversions
2181 · Counting Tilings
1653 · Elevator Rides

DATA STRUCTURES

3314 · Mountain Range

Task · source analysis length 3,364 characters

Problem statement

There are n mountains in a row, each with a specific height. You begin your hang gliding route from some mountain.

You can glide from mountain a to mountain b if mountain a is taller than mountain b and all mountains between a and b.

What is the maximum number of mountains you can visit on your route?

Input

The first line has an integer n: the number of mountains.

The next line has n integers $h_1, h_2, \ldots, h_n$: the heights of the mountains.

Output:

Print one integer: the maximum number of mountains.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le h_i \le 10^9$

Solution 1 (O(n log n))

We process the mountains in order of decreasing height and calculate the maximum length of a route that ends at each mountain.

For each mountain, we find the nearest higher mountain to the left and to the right. Since the mountains are processed in order of decreasing height, the maximum route length has already been calculated for both of these mountains. The longest route to the current mountain can be created by choosing the longest route to the left or right mountain and adding the current mountain to it.

There may be several mountains with the same height which makes the situation more complex. For this reason, we maintain an additional list of updates and apply the updates whenever the current height differs from the previous height.

```
#include <algorithm>
#include <iostream>
#include <map>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<pair<int, int>> heights;
    for (int i = 1; i <= n; i++) {
        int h;
        cin >> h;
        heights.emplace_back(h, i);
    }
    sort(heights.rbegin(), heights.rend());

    map<int, int> counts;
    counts[0] = 0;
    counts[n + 1] = 0;

    int prev_height = 0;
    vector<pair<int, int>> updates;
    int result = 0;

    for (auto [height, pos] : heights) {
        if (height != prev_height) {
            for (auto [pos, count] : updates) {
```

```
                counts[pos] = count;
            }
            updates.clear();
            prev_height = height;
        }

        auto it = counts.upper_bound(pos);
        int next_pos = it->first;
        it--;
        int prev_pos = it->first;

        int new_count = max(counts[prev_pos], counts[next_pos]) + 1;
        updates.emplace_back(pos, new_count);
        result = max(result, new_count);
    }

    cout << result << "\n";
}
```

Solution 2 (O(n))

It is also possible to solve the problem in linear time by using a different method for processing the mountains.

First, we find the nearest higher mountain to the left and to the right for each mountain, which can be done in linear time using stacks. We then create a graph that determines the order in which the mountains should be processed.

Next, we process the graph using topological sorting and calculate the maximum route length for each mountain, applying the same idea as in the previous solution.

```
#include <iostream>
#include <vector>
using namespace std;
const int INF = 1000000001;

int main() {
    int n;
    cin >> n;

    vector<int> heights(n + 2);
    heights[0] = INF;
    heights[n + 1] = INF;
    for (int i = 1; i <= n; i++) {
        cin >> heights[i];
    }

    vector<vector<int>> after(n + 2);

    vector<int> left_stack;
    left_stack.push_back(0);
    for (int i = 1; i <= n; i++) {
        while (heights[i] >= heights[left_stack.back()]) {
            left_stack.pop_back();
        }
        after[left_stack.back()].push_back(i);
        left_stack.push_back(i);
    }

    vector<int> right_stack;
    right_stack.push_back(n + 1);
    for (int i = n; i >= 1; i--) {
        while (heights[i] >= heights[right_stack.back()]) {
            right_stack.pop_back();
        }
        after[right_stack.back()].push_back(i);
        right_stack.push_back(i);
    }

    vector<int> count(n + 2);
    vector<int> result(n + 2);

    vector<int> queue;
    queue.push_back(0);
    queue.push_back(n + 1);

    int answer = 0;
    for (int i = 0; i < queue.size(); i++) {
        int pos = queue[i];
        for (auto next_pos : after[pos]) {
            result[next_pos] = max(result[next_pos], result[pos] + 1);
            answer = max(answer, result[next_pos]);
            count[next_pos]++;
```

```
            if (count[next_pos] == 2) {
                queue.push_back(next_pos);
            }
        }
    }
    cout << answer << "\n";
}
```

3304 · Visible Buildings Queries

Task · source analysis length 1,324 characters

Problem statement

There are n buildings in a row numbered $1, 2, \ldots, n$ from left to right. You are standing to the left of the first building. You can see a building if it is taller than all buildings to its left.

Your task is to process q queries: If only buildings in range $[a, b]$ existed, how many buildings would you see?

Input

The first line has two integers n and q: the number of buildings and queries.

The second line has n integers $h_1, h_2, \ldots, h_n$: the heights of the buildings.

Finally, there are q lines describing the queries. Each line has two integers a and b.

Output

For each query, print one integer: the number of visible buildings.

Constraints

- $1 \le n \le 10^5$
- $1 \le q \le 2 \cdot 10^5$
- $1 \le h_i \le 10^9$
- $1 \le a \le b \le n$

We first find the index of the next higher building for each building (or ∞ if there is no such building). This is done in linear time using a stack.

After that, we create a jump array that, for each building, indicates the next higher building to the right after x steps where x is a power of two. Since there are $O(\log n)$ possible powers of two, creating the array takes $O(n \log n)$ time.

Finally, we can answer any query in $O(\log n)$ time using the jump array.

```
#include <iostream>
#include <vector>
using namespace std;
const int N = 17;
const int INF = 1000000001;

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> heights(n + 2);
    for (int i = 1; i <= n; i++) {
        cin >> heights[i];
    }
    heights[n + 1] = INF;

    vector<vector<int>> jump(N, vector<int>(n + 2));
    jump[0][n + 1] = n + 1;

    vector<int> stack;
    stack.push_back(n + 1);
    for (int i = n; i >= 1; i--) {
        while (heights[stack.back()] <= heights[i]) {
            stack.pop_back();
        }
        jump[0][i] = stack.back();
        stack.push_back(i);
    }

    for (int k = 1; k < N; k++) {
        for (int i = 1; i <= n + 1; i++) {
            jump[k][i] = jump[k - 1][jump[k - 1][i]];
        }
    }

    while (q--) {
        int a, b;
```

```
        cin >> a >> b;
        int pos = a;
        int answer = 1;
        for (int k = N - 1; k >= 0; k--) {
            if (jump[k][pos] <= b) {
                answer += 1 << k;
                pos = jump[k][pos];
            }
        }
        cout << answer << "\n";
    }
}
```

2184 · Missing Coin Sum Queries

Task · source analysis length 2,388 characters

Problem statement

You have n coins with positive integer values. The coins are numbered $1, 2, \ldots, n$.

Your task is to process q queries of the form: "if you can use coins $a \ldots b$, what is the smallest sum you cannot produce?"

Input

The first input line has two integers n and q: the number of coins and queries.

The second line has n integers $x_1, x_2, \ldots, x_n$: the value of each coin.

Finally, there are q lines that describe the queries. Each line has two values a and b: you can use coins $a \ldots b$.

Output

Print the answer for each query.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$
- $1 \le a \le b \le n$

We process each query in B steps where $B = \lfloor \log_2 10^9 \rfloor = 30$. In each step, $b = 0, 1, \ldots, B-1$, we consider all coins in the query range such that the most significant bit of the coin value is in position b.

For each query i, we maintain a value $\mathbf{result}[i]$, which indicates that we can produce all coin sums between 1 and $\mathbf{result}[i]$ using the coins. Initially $\mathbf{result}[i] = 0$ for each $i = 1, 2, \ldots, q$.

In each step, we update the result for each query. Let $\mathbf{sum}(i, b)$ and $\mathbf{min}(i, b)$ denote the sum and minimum of coin values in the range of query i whose most significant bit is in position b. Always when $\mathbf{min}(i, b) \le \mathbf{result}[i] + 1$, we can extend the range of possible sums by adding $\mathbf{sum}(i, b)$ to $\mathbf{result}[i]$. Finally, $\mathbf{result}[i] + 1$ is the smallest missing coin sum.

We use data structures for efficient range sum and range minimum queries. Using these data structures, we can process range sum queries in $O(1)$ time after $O(n)$ time preprocessing and range minimum queries in $O(1)$ time after $O(n \log n)$ time preprocessing.

The resulting algorithm works in $O(B(n \log n + q))$ time.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int INF = 1e9 + 1;
const int B = 30;
const int K = 18;

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> x(n + 1);
    vector<int> bit(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> x[i];
        bit[i] = 31 - __builtin_clz(x[i]);
    }

    vector<pair<int, int>> query(q + 1);
    for (int i = 1; i <= q; i++) {
        cin >> query[i].first >> query[i].second;
    }

    vector<ll> result(q + 1);
    vector<ll> sumq(n + 1);
    vector<vector<int>> minq(K, vector<int>(n + 1));
```

```
    for (int b = 0; b < B; b++) {
        for (int i = 1; i <= n; i++) {
            sumq[i] = sumq[i - 1];
            if (bit[i] == b) {
                sumq[i] += x[i];
            }
        }

        for (int i = 1; i <= n; i++) {
            minq[0][i] = bit[i] == b ? x[i] : INF;
        }
        for (int k = 1; k < K; k++) {
            for (int i = 1; i <= n - (1 << (k - 1)); i++) {
                minq[k][i] =
                    min(minq[k - 1][i], minq[k - 1][i + (1 << (k - 1))]);
            }
        }

        for (int i = 1; i <= q; i++) {
            auto [a, b] = query[i];
            int level = 31 - __builtin_clz(b - a + 1);
            int min_x = min(minq[level][a], minq[level][b - (1 << level) + 1]);
            if (min_x <= result[i] + 1) {
                result[i] += sumq[b] - sumq[a - 1];
            }
        }
    }

    for (int i = 1; i <= q; i++) {
        cout << result[i] + 1 << "\n";
    }
}
```

1077 · Sliding Window Cost

Task · source analysis length 3,946 characters

Problem statement

You are given an array of n integers. Your task is to calculate for each window of k elements, from left to right, the minimum total cost of making all elements equal.

You can increase or decrease each element with cost x where x is the difference between the new and the original value. The total cost is the sum of such costs.

Input

The first line contains two integers n and k: the number of elements and the size of the window.

Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output

Output $n - k + 1$ values: the costs.

Constraints

- $1 \le k \le n \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$

It is known that we can achieve a minimum cost inside a window if we set each element to be the median of the window. Thus, the remaining problem is how to efficiently calculate this for each window.

Solution 1

The following code uses two multisets `left` and `right` that contain the values inside a window: `left` contains the median and all element before the median, and `right` contains all elements after the median. In addition, `left_sum` and `right_sum` have the sum of elements in `left` and `right`.

When the window moves, we update the multisets and sums, and then we can calculate the minimum total cost using them. Each window move requires a fixed number of $O(\log n)$ operations, so the algorithm works in $O(n \log n)$ time.

```
#include <iostream>
#include <set>
#include <vector>
using namespace std;
using ll = long long;

int main() {
    int n, k;
    cin >> n >> k;
    vector<int> x(n);
    for (int i = 0; i < n; i++) {
        cin >> x[i];
    }

    multiset<int> left, right;
    ll left_sum = 0, right_sum = 0;

    auto fix = [&] {
        if (left.size() > right.size() + 1) {
            auto v = *left.rbegin();
            left_sum -= v;
            right_sum += v;
            right.insert(v);
            left.erase(left.find(v));
        }
        if (right.size() > left.size()) {
            auto v = *right.begin();
            right_sum -= v;
            left_sum += v;
            left.insert(v);
            right.erase(right.find(v));
        }
    };
```

```
    auto add = [&](int v) {
        if (!left.empty() && *left.rbegin() > v) {
            left.insert(v);
            left_sum += v;
        } else {
            right.insert(v);
            right_sum += v;
        }
        fix();
    };

    auto rem = [&](int v) {
        if (*left.rbegin() >= v) {
            left.erase(left.find(v));
            left_sum -= v;
        } else {
            right.erase(right.find(v));
            right_sum -= v;
        }
        fix();
    };

    auto cost = [&] {
        ll median = *left.rbegin();
        ll left_cost = left.size() * median - left_sum;
        ll right_cost = right_sum - right.size() * median;
        return left_cost + right_cost;
    };

    for (int i = 0; i < n; i++) {
        add(x[i]);
        if (i >= k) {
            rem(x[i - k]);
        }
        if (i >= k - 1) {
            cout << cost() << " ";
        }
    }
    cout << "\n";
}
```

Solution 2

Another way to solve the problem is to use segment trees. We first compress the input values so that they are in the range $[0, n-1]$. After that, we maintain two segment trees: `tree` maintains the count of each value and `sum_tree` maintains the sum of values.

We find the position of the median using `tree` and the sum of values before and after the median using `sum_tree`. Each window move requires a constant number of tree operations, so the algorithm works in $O(n \log n)$ time.

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;
using ll = long long;

const int N = 1 << 18;
int tree[N * 2];
ll sum_tree[N * 2];

// Adds x to position k in tree.
// Adds y to position k in sum_tree.
void change(int k, int x, int y) {
    k += N;
    while (k > 0) {
        tree[k] += x;
        sum_tree[k] += y;
        k /= 2;
    }
}

// Returns the sum of the values in range [l, r] in sum_tree.
ll query(int l, int r) {
    l += N;
    r += N;
    ll sum = 0;
    while (l <= r) {
        if (l % 2 == 1) sum += sum_tree[l++];
        if (r % 2 == 0) sum += sum_tree[r--];
```

```
        l /= 2;
        r /= 2;
    }
    return sum;
}

// Returns the first position in tree where prefix sum is at least sum.
int search(int sum) {
    int s = 1;
    while (s < N) {
        if (tree[s * 2] >= sum) {
            s = s * 2;
        } else {
            sum -= tree[s * 2];
            s = s * 2 + 1;
        }
    }
    return s - N;
}

int main() {
    int n, k;
    cin >> n >> k;

    vector<pair<int, int>> pairs;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        pairs.emplace_back(x, i);
    }

    sort(pairs.begin(), pairs.end());
    vector<int> pos(n);
    for (int i = 0; i < n; i++) {
        pos[pairs[i].second] = i;
    }

    for (int i = 0; i < n; i++) {
        change(pos[i], 1, pairs[pos[i]].first);

        if (i >= k) {
            change(pos[i - k], -1, -pairs[pos[i - k]].first);
        }

        if (i >= k - 1) {
            int m = (k + 1) / 2;
            int p = search(m);
            int median = pairs[p].first;

            ll left_cost = ll(m) * median - query(0, p);
            ll right_cost = query(p + 1, n) - ll(k - m) * median;

            cout << left_cost + right_cost << " ";
        }
    }
    cout << "\n";
}
```

STRING

3359 · Minimal Grid Path

Task · source analysis length 1,549 characters

Problem statement

You are given an $n \times n$ grid whose each square contains a letter.

You should move from the upper-left square to the lower-right square. You can only move right or down.

What is the lexicographically minimal string you can construct?

Input

The first line has an integer n: the size of the grid.

After this, there are n lines that describe the grid. Each line has n letters between A and Z.

Output

Print the lexicographically minimal string.

Constraints

- $1 \le n \le 3000$

The algorithm consists of $2n - 2$ rounds that construct the minimal string corresponding to a path from the top-left square to the bottom-right square. It maintains a prefix of this minimal string throughout the process.

In each round, we examine all possible ending positions of paths that correspond to the current prefix. The prefix is extended by adding a new letter. From each ending position, we consider moving one step to the right and one step down to determine the next letter to add to the path.

It is important to ensure that the list of ending positions contains only distinct positions after each round. Otherwise, the algorithm could process the same ending positions multiple times, making it inefficient.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<string> grid(n);
    for (int i = 0; i < n; i++) {
        cin >> grid[i];
    }

    cout << grid[0][0];

    vector<pair<int, int>> level;
    level.emplace_back(0, 0);

    for (int i = 0; i < 2 * n - 2; i++) {
        vector<pair<int, int>> next_level;
        char min_letter = 'Z' + 1;

        auto update_next_level = [&](int y, int x) {
            if (y < n && x < n) {
                char letter = grid[y][x];
                if (letter < min_letter) {
                    next_level.clear();
                    min_letter = letter;
                }
                if (letter == min_letter) {
                    next_level.emplace_back(y, x);
                }
            }
        };

        for (auto [y, x] : level) {
            update_next_level(y + 1, x);
            update_next_level(y, x + 1);
        }
```

```
        cout << min_letter;

        level = next_level;
        level.erase(unique(level.begin(), level.end()), level.end());
    }
    cout << "\n";
}
```

2186 · Special Substrings

Task · source analysis length 1,822 characters

Problem statement

A substring is called *special* if every character that appears in the string appears the same number of times in the substring.

Your task is to count the number of special substrings in a given string.

Input

The only input line has a string of length n. Every character is between a...z.

Output

Print one integer: the number of special substrings.

Constraints

- $1 \le n \le 2 \cdot 10^5$

In this analysis, we assume that each character from $\mathtt{a}$ to $\mathtt{z}$ appears in the string. If this is not the case, some extra work is required to ignore the missing characters.

Let $C_{i,c}$ be the number of occurrences of character c among the first i characters. Now, character c appears in the substring of characters at indices $[i, j)$ exactly $C_{j,c} - C_{i,c}$ times.

For a substring to be special, every character needs to appear a constant amount of times. We're not interested in the specific constant, which means we can keep a state of character occurrences relative to some reference character, say $\mathtt{a}$:

$$\begin{aligned}(&C_{i,\mathtt{a}} - C_{i,\mathtt{a}},\\ &C_{i,\mathtt{b}} - C_{i,\mathtt{a}},\\ &\dots,\\ &C_{i,\mathtt{z}} - C_{i,\mathtt{a}}),\end{aligned}$$

Let us look at any character x. Note that if we add a constant amount of occurrences of both x and the reference character $\mathtt{a}$, the relative count stays constant, which means it suffices to count the number of previous occurrences of the same state.

We will store the current state in an `array<int, 26>` and previous states to a `map`. The time complexity of a map operation is $O(\log n \cdot \Sigma)$, where Σ represents the alphabet size. The time complexity of the complete solution is $O(n \log n \cdot \Sigma)$.

```
#include <array>
#include <iostream>
#include <map>
using namespace std;
using ll = long long;

using A = array<int, 26>;

int main() {
    string s;
    cin >> s;

    bool seen[26]{};
    char reference_char = s[0];

    for (char c : s) seen[c - 'a'] = true;

    map<A, int> m;
    A cur{};
    m[cur] = 1;
    ll ans = 0;

    for (char c : s) {
        if (c == reference_char) {
            for (int i = 0; i < 26; ++i) {
```

```
                if (seen[i]) cur[i]--;
            }
        }
        cur[c - 'a']++;
        ans += m[cur];
        m[cur]++;
    }

    cout << ans << '\n';
}
```

1743 · String Reorder

Task · source analysis length 1,351 characters

Problem statement

Your task is to reorder the characters of a string so that no two adjacent characters are the same. What is the lexicographically minimal such string?

Input

The only line has a string of length n consisting of characters A–Z.

Output

Print the lexicographically minimal reordered string where no two adjacent characters are the same. If it is not possible to create such a string, print -1.

Constraints

- $1 \le n \le 10^6$

Let us first analyze when is it possible construct a string such that no two adjacent characters are the same. The maximum amount of occurrences for a single character is reached when it appears in every other position, including the first and the last. This leads to the inequality $2k \le n + 1$, where k is the number of occurrences of the most common character and n is the length of the string. The construction is possible if and only if the above inequality holds.

Now, we may construct the answer, one character at a time. To get the lexicographically smallest string, we will, at each step, choose the smallest character greedily. The above condition is used to check if the string can be completed by choosing a specific character.

```
#include <iostream>
using namespace std;

int count[128];

int main() {
    string s;
    cin >> s;

    int n = s.size();

    for (auto c : s) {
        count[(int)c]++;
    }

    auto check = [&](int length) {
        for (int i = 'A'; i <= 'Z'; ++i) {
            if (count[i] * 2 > length + 1) {
                return false;
            }
        }
        return true;
    };

    if (!check(n)) {
        cout << -1 << '\n';
        return 0;
    }

    char prev = '$';
    for (int i = 1; i <= n; ++i) {
        for (int nxt = 'A'; nxt <= 'Z'; ++nxt) {
            if (!count[nxt]) continue;
            if (nxt == prev) continue;
            count[nxt]--;
            if (check(n - i)) {
                cout << (char)nxt;
                prev = nxt;
                break;
            }
            count[nxt]++;
        }
    }
```

```
    cout << '\n';
}
```

1112 · Required Substring

Task · source analysis length 3,866 characters

Problem statement

Your task is to calculate the number of strings of length n having a given pattern of length m as their substring. All strings consist of characters A–Z.

Input

The first input line has an integer n: the length of the final string.

The second line has a pattern of length m.

Output

Print the number of strings modulo $10^9 + 7$.

Constraints

- $1 \le n \le 1000$
- $1 \le m \le 100$

Solution 1

Suppose we read characters of a string one after another and have to detect when the last m characters form the pattern. To do that, we have to remember the longest prefix of the pattern that is also a suffix of our current string.

For example, if the pattern is ABABC and we have read the string CAABAB, the prefix is ABAB (length 4). Consider what can happen when we read the next character:

- If we read A, the prefix becomes ABA (length 3).
- If we read B, the prefix becomes empty (length 0).
- If we read C, the prefix becomes ABABC (length 5).
- If we read any other character, the prefix becomes empty (length 0).

We define a transition function $\text{step}[i][c]$: if the length of the prefix is i and we read character c, what is the length of the prefix after that? For example, in the above situation $\text{step}[4][\text{A}] = 3$, $\text{step}[4][B] = 0$, and $\text{step}[4][\text{C}] = 5$. We also define that $\text{step}[m][c] = m$ for any character c because in that case the pattern is a subtring of the string, regardless of what characters will come after that.

After that, we use dynamic programming to solve the problem. We calculate values of $\text{ways}[i][j]$: in how many ways can we create a string of length i that ends with a prefix whose length is j (or contains the whole pattern if $j = m$)?

The answer to the problem is $\text{ways}[n][m]$, and the algorithm works in $O(nmk)$ time where k is the size of the alphabet.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

int step[101][128];
ll ways[1001][101];

int main() {
    int n;
    cin >> n;
    string s;
    cin >> s;
    int m = s.size();

    for (int i = 0; i < m; i++) {
        for (int c = 'A'; c <= 'Z'; c++) {
            string u = s.substr(0, i) + (char)c;
            while (s.substr(0, u.size()) != u) {
                u = u.substr(1);
            }
```

```
            step[i][c] = u.size();
        }
    }
    for (int c = 'A'; c <= 'Z'; c++) {
        step[m][c] = m;
    }

    ways[0][0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= m; j++) {
            for (int c = 'A'; c <= 'Z'; c++) {
                ways[i + 1][step[j][c]] += ways[i][j];
                ways[i + 1][step[j][c]] %= M;
            }
        }
    }

    cout << ways[n][m] << '\n';
}
```

Solution 2: prefix function

The construction of the finite state machine can be sped up by utilizing the prefix function. It allows us to construct the transitions in $O(nk)$ time.

The prefix function $\pi(i)$ of a string is defined as follows: each $\pi(i)$ denotes the maximum length of a substring that ends at position i and is a strict prefix of the string. The prefix function might be familiar from the KMP algorithm.

```
#include <algorithm>
#include <array>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

// Returns the values of the prefix function for string s.
// The prefix function pi(i) is defined as follows: each pi(i) denotes the
// maximum length of a substring that ends at position i and is a strict prefix
// of the string.
vector<int> prefix_function(string &s) {
    const int n = (int)s.size();
    vector<int> r(n);
    for (int i = 1; i < n; ++i) {
        int j = r[i - 1];
        while (j > 0 && s[i] != s[j]) j = r[j - 1];
        r[i] = j + (s[i] == s[j]);
    }
    return r;
}

int step[101][128];
ll ways[1001][101];

int main() {
    int n;
    cin >> n;

    string s;
    cin >> s;

    int m = s.size();
    auto links = prefix_function(s);

    for (int i = 0; i < m; i++) {
        for (int c = 'A'; c <= 'Z'; c++) {
            if (s[i] == c) {
                step[i][c] = i + 1;
            } else if (i > 0) {
                step[i][c] = step[links[i - 1]][c];
            }
        }
    }
    for (int c = 'A'; c <= 'Z'; c++) {
        step[m][c] = m;
    }

    ways[0][0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= m; j++) {
            for (int c = 'A'; c <= 'Z'; c++) {
```

```
                ways[i + 1][step[j][c]] += ways[i][j];
                ways[i + 1][step[j][c]] %= M;
            }
        }
    }

    cout << ways[n][m] << '\n';
}
```

References

- Wikipedia (Knuth–Morris–Pratt algorithm)

1087 · Shortest Subsequence

Task · source analysis length 2,321 characters

Problem statement

You are given a DNA sequence consisting of characters A, C, G, and T.

Your task is to find the shortest DNA sequence that is not a subsequence of the original sequence.

Input

The only input line contains a DNA sequence with n characters.

Output

Print the shortest DNA sequence that is not a subsequence of the original sequence. If there are several solutions, you may print any of them.

Constraints

- $1 \le n \le 10^6$

We will present a greedy approach and another, simplified version of the same algorithm.

Start by creating a two dimensional array. Populate this array for each character and position with the next position at which the character appears in the string. With this array, we may check if a subsequence appears in the string. This may be done by going through the subsequence one character at a time and jumping to the next occurrence of the character. If at any point there is no such occurrence, then the string is not a subsequence of the original string.

If the original string contains all of the four characters, the answer will always have a prefix that is a subsequence of the string. A prefix of the optimal length may be found with a greedy algorithm, that is, we always add the character whose next occurrence has the greatest index. It is always optimal to jump as far as possible because the occurrence arrays are increasing. Whenever one of the characters has no next occurrence, we add the character to the string and have an answer.

```
#include <algorithm>
#include <iostream>
using namespace std;

const int N = 1e6;
const int INF = 1e9;

int nxt[N + 1][4];

int main() {
    string s;
    cin >> s;

    int n = s.size();

    for (int j = 0; j < 4; ++j) {
        nxt[n][j] = INF;
    }

    for (int i = n; i > 0; --i) {
        for (int j = 0; j < 4; ++j) {
            nxt[i - 1][j] = nxt[i][j];
        }
        int idx = string("ACGT").find(s[i - 1]);
        nxt[i - 1][idx] = i;
    }

    int i = 0;
    string ans;
    while (i < INF) {
        int c = max_element(nxt[i], nxt[i] + 4) - nxt[i];
        ans += "ACGT"[c];
        i = nxt[i][c];
    }

    cout << ans << '\n';
}
```

This solution may be greatly simplified. Remember that, at any time, the next character picked will be the one whose next occurrence has the greatest index. Now, this character may be found by maintaining a set of seen characters. Whenever we have seen three of the characters and come across the fourth one, we know the fourth one is the one with the greatest distance from the index of the last picked character.

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    string s;
    cin >> s;

    string ans;
    set<char> seen;

    for (char c : s) {
        seen.insert(c);
        if (seen.size() == 4) {
            ans += c;
            seen.clear();
        }
    }

    for (char c : "ACGT") {
        if (!seen.count(c)) {
            ans += c;
            break;
        }
    }

    cout << ans << '\n';
}
```

GRAPH

2177 · Strongly Connected Edges

Task · source analysis length 1,665 characters

Problem statement

Given an undirected graph, your task is to choose a direction for each edge so that the resulting directed graph is strongly connected.

Input

The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$.

After this, there are m lines describing the edges. Each line has two integers a and b: there is an edge between nodes a and b.

You may assume that the graph is simple, i.e., there are at most one edge between two nodes and every edge connects two distinct nodes.

Output

Print m lines describing the directions of the edges. Each line has two integers a and b: there is an edge from node a to node b. You can print any valid solution.

If there are no solutions, only print `IMPOSSIBLE`.

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le a, b \le n$

It is useful to approach this problem by focusing on a DFS tree of the undirected graph. The DFS contains *tree edges* and *back edges*.

We will direct each tree edge from the parent to the child. Further, let each *back edge* run upwards towards the root. Turns out this construction always works if it is possible to make the graph strongly connected overall.

There are two cases in which it is not possible to make the graph strongly connected. Firstly, it is not possible if the given graph is not connected. Secondly, if there is a subtree in the DFS tree from which there is not a single back-edge going outside of it, it is also not possible, because there is no way to reach other nodes from the subtree.

```
#include <iostream>
#include <vector>
using namespace std;

void fail() {
    cout << "IMPOSSIBLE" << '\n';
    exit(0);
}

const int N = 100001;
vector<int> g[N];
vector<pair<int, int>> ans;
bool visited[N];
int depth[N], min_depth[N];

void dfs(int node, int prev) {
    visited[node] = true;
    min_depth[node] = depth[node];
    for (int nxt : g[node]) {
        if (nxt == prev) continue;
        if (visited[nxt]) {
            if (depth[nxt] < depth[node]) {
                ans.emplace_back(node, nxt);
            }
            min_depth[node] = min(min_depth[node], depth[nxt]);
        } else {
            depth[nxt] = depth[node] + 1;
            dfs(nxt, node);
            min_depth[node] = min(min_depth[node], min_depth[nxt]);
```

```
            ans.emplace_back(node, nxt);
            if (min_depth[nxt] > depth[node]) fail();
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;

    for (int i = 0; i < m; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1);

    if (ans.size() != m) fail();

    for (auto [a, b] : ans) {
        cout << a << ' ' << b << '\n';
    }
}
```

2143 · Reachability Queries

Task · source analysis length 2,671 characters

Problem statement

A directed graph consists of n nodes and m edges. The edges are numbered $1, 2, \ldots, n$.

Your task is to answer q queries of the form "can you reach node b from node a?"

Input

The first input line has three integers n, m and q: the number of nodes, edges and queries.

Then there are m lines describing the edges. Each line has two distinct integers a and b: there is an edge from node a to node b.

Finally there are q lines describing the queries. Each line consists of two integers a and b: "can you reach node b from node a?"

Output

Print the answer for each query: either "YES" or "NO".

Constraints

- $1 \le n \le 5 \cdot 10^4$
- $1 \le m, q \le 10^5$

It is useful to think of the problem in terms of strongly connected components. In particular, if two nodes are in the same in the same component, they are both reachable from each other.

The strongly connected components of the graph also form a directed acyclic graph (DAG) of their own. There cannot be cycles in the graph condensed from the components because if it contained a cycle of multiple components, the nodes of the cycle would, by definition, form only a single component.

Once we have found the strongly connected components using Tarjan's algorithm, we have reduced the problem to a DAG. Because there are no cycles in a DAG, we can make a dynamic programming solution. In particular, we may store, for each node, information about which other nodes are reachable from it. The reachability information can be copied to each *parent* of a node, once all the children of the node have been fully processed.

If we were to store the reachability information in a boolean array, the solution would turn out too slow. We may, however, use bitsets to speed up the transformation of the information as bitsets allow us to update 64 elements at once.

```
#include <bitset>
#include <iostream>
#include <vector>
using namespace std;

const int N = 5e4 + 1;
vector<int> g[N];
int cur_depth, comp_cnt;
int comp[N], depth[N], low[N];
vector<int> stack;
bool in_stack[N];

// Tarjan's strongly connected components algorithm
void scc_dfs(int node) {
    low[node] = depth[node] = ++cur_depth;
    stack.push_back(node);
    in_stack[node] = true;
    for (int nxt : g[node]) {
        if (!depth[nxt]) {
            scc_dfs(nxt);
            low[node] = min(low[node], low[nxt]);
        } else if (in_stack[nxt]) {
            low[node] = min(low[node], depth[nxt]);
        }
    }
    if (depth[node] == low[node]) {
        int i = -1;
        ++comp_cnt;
        while (i != node) {
```

```
            i = stack.back();
            stack.pop_back();
            in_stack[i] = false;
            comp[i] = comp_cnt;
        }
    }
}

using B = bitset<N>;
// `cg` is a condensed graph constructed from the SCC's.
vector<int> cg[N];
B reach[N];
bool visited[N];

void dfs(int node) {
    if (visited[node]) return;
    visited[node] = true;
    reach[node][node] = true;
    for (int nxt : cg[node]) {
        dfs(nxt);
        reach[node] |= reach[nxt];
    }
}

int main() {
    int n, m, q;
    cin >> n >> m >> q;

    for (int i = 1; i <= m; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
    }

    for (int i = 1; i <= n; ++i) {
        if (!depth[i]) scc_dfs(i);
    }

    for (int i = 1; i <= n; ++i) {
        for (int j : g[i]) {
            if (comp[i] != comp[j]) {
                cg[comp[i]].push_back(comp[j]);
            }
        }
    }

    for (int i = 1; i <= comp_cnt; ++i) {
        dfs(i);
    }

    for (int qi = 1; qi <= q; ++qi) {
        int a, b;
        cin >> a >> b;
        cout << (reach[comp[a]][comp[b]] ? "YES" : "NO") << '\n';
    }
}
```

2138 · Reachable Nodes

Task · source analysis length 1,376 characters

Problem statement

A directed acyclic graph consists of n nodes and m edges. The nodes are numbered $1, 2, \ldots, n$.

Calculate for each node the number of nodes you can reach from that node (including the node itself).

Input

The first input line has two integers n and m: the number of nodes and edges.

Then there are m lines describing the edges. Each line has two distinct integers a and b: there is an edge from node a to node b.

Output

Print n integers: for each node the number of reachable nodes.

Constraints

- $1 \le n \le 5 \cdot 10^4$
- $1 \le m \le 10^5$

A directed acyclic graph does not contain cycles which means that we may use a dynamic programming approach to solve the problem. Let us store a boolean array for each node, which tells us which nodes are reachable from the node. A straightforward solution would perform a DFS from each node and update the arrays after each edge. Each edge would correspond to $O(n)$ operations and thus, the solution would result in a time complexity of $O(nm)$ and would perform roughly $5 \cdot 10^4 \cdot 10^5 = 5 \cdot 10^9$ operations, which makes the solution a bit too slow.

Fortunately, the approach can be sped up using bitsets. Bitsets allow us to update 64 positions of the array at once and thus, provide a factor of $1/64$ to the number of operations, bringing the total down to the order of $5 \cdot 10^4 \cdot 10^5 \cdot \frac{1}{64} \approx 7.8 \cdot 10^7$.

```
#include <bitset>
#include <iostream>
#include <vector>
using namespace std;

const int N = 5e4 + 1;

vector<int> g[N];
bitset<N> reach[N];
bool visited[N];

void dfs(int node) {
    if (visited[node]) return;
    visited[node] = true;
    reach[node][node] = true;
    for (int nxt : g[node]) {
        dfs(nxt);
        reach[node] |= reach[nxt];
    }
}

int main() {
    int n, m;
    cin >> n >> m;

    for (int i = 1; i <= m; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
    }

    for (int i = 1; i <= n; ++i) {
        dfs(i);
        cout << reach[i].count() << ' ';
    }
    cout << '\n';
}
```

2080 · Fixed-Length Paths I

Task · source analysis length 7,847 characters

Problem statement

Given a tree of n nodes, your task is to count the number of distinct paths that consist of exactly k edges.

Input

The first input line contains two integers n and k: the number of nodes and the path length. The nodes are numbered $1, 2, \ldots, n$.

Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output

Print one integer: the number of paths.

Constraints

- $1 \le k \le n \le 2 \cdot 10^5$
- $1 \le a, b \le n$

Solution 1: Centroid Decomposition

This problem can be solved with a technique known as centroid decomposition.

The idea is to recursively split the tree into disjoint trees. We start by selecting a node from the tree. Then, we calculate the number of paths of length k which go through the selected node. After this, all unaccounted paths are completely contained in the subtrees of the node. Thus, we may remove the selected node and repeat the same procedure for the resulting new trees.

The solution would be too slow if we each time selected the node at random. The key idea is to pick the node so that it is a centroid of the tree, i.e., a node such that when it is appointed the root of the tree, each subtree has at most $\lfloor n/2 \rfloor$ nodes.

If we choose a centroid node, the sizes of the resulting trees will be at most half of the original size. This means that each node will be part of at most $O(\log n)$ trees. As every node of every tree is processed once, the total time complexity of the solution is $O(n \log n)$.

At each step, a centroid node can be found with a greedy algorithm, which works as follows: root the tree, traverse recursively from the root, each time choosing a child with a subtree size of at least $\lfloor n/2 \rfloor$. If there is no such child, the traversal has arrived at a centroid node.

When we have found a centroid, we can start counting the paths. For this purpose, we use an array `dist`. Each position i of `dist` stores the amount of nodes at distance i from the centroid. To ensure that the counted paths are not contained solely in a single subtree and that each path is only counted once, we count the number of paths ending in a subtree before updating the array `dist` with the distances of the nodes in the subtree.

The `dist` array is reused for all of the individually processed trees. So, we have to reset it between the recursive calls.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int N = 200001;
bool deleted[N];
int n, k, sizes[N];
vector<int> g[N];
ll answer;

void dfs_sizes(int node, int parent) {
    sizes[node] = 1;
    for (int child : g[node]) {
        if (child == parent || deleted[child]) continue;
        dfs_sizes(child, node);
        sizes[node] += sizes[child];
```

```
    }
}

int find_centroid(int node, int parent, int n) {
    for (int child : g[node]) {
        if (child != parent && !deleted[child] && sizes[child] > n / 2) {
            return find_centroid(child, node, n);
        }
    }
    return node;
}

int dist[N];

void dfs_add(int node, int parent, int depth, int change) {
    dist[depth] += change;
    for (int child : g[node]) {
        if (child == parent || deleted[child]) continue;
        dfs_add(child, node, depth + 1, change);
    }
}

void dfs_query(int node, int parent, int depth) {
    if (k - depth >= 0) answer += dist[k - depth];
    for (int child : g[node]) {
        if (child == parent || deleted[child]) continue;
        dfs_query(child, node, depth + 1);
    }
}

void recurse(int node) {
    dfs_sizes(node, -1);
    int r = find_centroid(node, -1, sizes[node]);
    dist[0] = 1;
    for (int child : g[r]) {
        if (!deleted[child]) {
            dfs_query(child, r, 1);
            dfs_add(child, r, 1, 1);
        }
    }
    dfs_add(r, -1, 0, -1);
    deleted[r] = true;
    for (int child : g[r]) {
        if (!deleted[child]) {
            recurse(child);
        }
    }
}

int main() {
    cin >> n >> k;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    recurse(1);
    cout << answer << '\n';
}
```

Solution 2: Small-To-Large Merging

There is another, simpler solution to this problem. It uses a technique sometimes known as *small-to-large merging*. The implementation has only one tree traversal.

After rooting the tree, we make a depth-first traversal of it. During the traversal, we calculate the number of paths through each node. Any path in a tree has a highest node: a node that is the closest to the root. We will calculate each path only once, at its highest node.

To make sure we only count paths of length k, we use the fact that the distance between tree nodes a and b equals

$$\text{depth}(a) + \text{depth}(b) - 2 \cdot \text{depth}(c),$$

where c is the highest node on the path between a and b, also known as the lowest common ancestor of a and b. With this, finding the number of paths of length k through node x corresponds to finding the number of ways to pick two nodes a and b from different subtrees of x with the following condition:

$$\text{depth}(a) + \text{depth}(b) - 2 \cdot \text{depth}(x) = k.$$

We create a `map` for each node, which stores the depths of the nodes in the subtree of the node. We use the technique of *small-to-large* merging for merging a node's `map` to its parent's `map` efficiently: if a map of a child is larger than the map of its parent, we swap the data structures before merging. This means that instead of moving elements from the map of the child to the map of its parent, we move them from the smaller map to the larger map.

It can be proven that each element will be moved at most $O(\log n)$ times making the total time complexity of the solution $O(n \log^2 n)$.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;
using ll = long long;

const int N = 200001;
int n, k;
vector<int> g[N];
ll answer;

map<int, int> dfs(int node, int parent, int depth) {
    map<int, int> dist;
    dist[depth] = 1;
    for (int child : g[node]) {
        if (child == parent) continue;
        auto child_dist = dfs(child, node, depth + 1);
        if (child_dist.size() > dist.size()) {
            swap(dist, child_dist);
        }
        for (auto [d, amt] : child_dist) {
            answer += 1ll * dist[k - d + 2 * depth] * amt;
        }
        for (auto [d, amt] : child_dist) {
            dist[d] += amt;
        }
    }
    return dist;
}

int main() {
    cin >> n >> k;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1, 0);
    cout << answer << '\n';
}
```

Solution 3: Improved merge

Instead of using a `map` to store information about the depths of the nodes in each subtree, we use a `deque`. Other than that, this solution works in a way that is very similar to the previous solution.

Note that, in contrast to the previous solution, the `deque` indexes represent depths relative to the current node, as opposed to relative to the root node. Also note that the size of the merged data structure is proportional to the depth of the node, instead of the subtree size of the node.

It may come as a surprise that this solution runs in $O(n)$ time.

This proof was first presented by Codeforces user neal.

The time complexity of constructing the `deque`s is linear because each insertion works in $O(1)$ time and each node corresponds to at most two insertions. Thus, we need to focus only on the `merge` function.

A `merge` operation results in a `deque` with a size equal to the size of the larger of the merged `deque`s, as opposed to the sum of the two sizes. Thus, the smaller `deque` is "consumed" and its elements will not amount to any more work. As there are only $O(n)$ `deque` elements in total and each of them is consumed at most once, the total time complexity is linear.

```
#include <deque>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int N = 200001;
int n, k;
vector<int> g[N];
ll answer;

void merge(deque<int> &a, deque<int> &b) {
    if (b.size() > a.size()) swap(a, b);
    int bs = b.size();
    for (int i = 0; i < bs; ++i) {
        if (0 <= k - i && k - i < (int)a.size()) {
            answer += b[i] * a[k - i];
        }
    }
    for (int i = 0; i < bs; ++i) {
        a[i] += b[i];
    }
}

deque<int> dfs(int node, int parent) {
    deque<int> ret{1};
    for (int child : g[node]) {
        if (child == parent) continue;
        auto child_depths = dfs(child, node);
        child_depths.push_front(0);
        merge(ret, child_depths);
    }
    return ret;
}

int main() {
    cin >> n >> k;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1);
    cout << answer << '\n';
}
```

1686 · Coin Collector

Task · source analysis length 2,243 characters

Problem statement

A game has n rooms and m tunnels between them. Each room has a certain number of coins. What is the maximum number of coins you can collect while moving through the tunnels when you can freely choose your starting and ending room?

Input

The first input line has two integers n and m: the number of rooms and tunnels. The rooms are numbered $1, 2, \ldots, n$.

Then, there are n integers $k_1, k_2, \ldots, k_n$: the number of coins in each room.

Finally, there are m lines describing the tunnels. Each line has two integers a and b: there is a tunnel from room a to room b. Each tunnel is a one-way tunnel.

Output

Print one integer: the maximum number of coins you can collect.

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le k_i \le 10^9$
- $1 \le a, b \le n$

We first identify the strongly connected components and create a component graph where each node's value is the total number of coins in the component. We use Kosaraju's algorithm for finding the components.

After that, we find the maximum coin sum in a path in the component graph. We can use dynamic programming to find the maximum sum because the component graph is a directed acyclic graph. In the dynamic programming implementation, we use the fact that Kosaraju's algorithm creates the components in reverse topological order.

The resulting algorithm works in $O(n + m)$ time.

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;
using ll = long long;

vector<vector<int>> graph;
vector<vector<int>> graph_rev;

vector<int> coins;

vector<int> order;
vector<int> seen;

void find_order(int node) {
    if (seen[node]) return;
    seen[node] = 1;
    for (auto next_node : graph[node]) {
        find_order(next_node);
    }
    order.push_back(node);
}

vector<int> component;
vector<vector<int>> nodes;
vector<ll> total_coins;

void find_component(int node, int id) {
    if (component[node]) return;
    component[node] = id;
    nodes[id].push_back(node);
    total_coins[id] += coins[node];
    for (auto next_node : graph_rev[node]) {
```

```
        find_component(next_node, id);
    }
}

int main() {
    int n, m;
    cin >> n >> m;

    coins.resize(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> coins[i];
    }

    graph.resize(n + 1);
    graph_rev.resize(n + 1);

    for (int i = 1; i <= m; i++) {
        int a, b;
        cin >> a >> b;
        graph[a].push_back(b);
        graph_rev[b].push_back(a);
    }

    seen.resize(n + 1);
    for (int node = 1; node <= n; node++) {
        find_order(node);
    }

    reverse(order.begin(), order.end());
    int count = 0;
    component.resize(n + 1);
    nodes.resize(n + 1);
    total_coins.resize(n + 1);
    for (auto node : order) {
        if (component[node]) continue;
        count++;
        find_component(node, count);
    }

    vector<ll> max_sum(count + 1);
    ll result = 0;
    for (int id = count; id >= 1; id--) {
        ll best_sum = 0;
        for (auto node : nodes[id]) {
            for (auto next_node : graph[node]) {
                if (component[next_node] != id) {
                    best_sum = max(best_sum, max_sum[component[next_node]]);
                }
            }
        }
        max_sum[id] = total_coins[id] + best_sum;
        result = max(result, max_sum[id]);
    }
    cout << result << "\n";
}
```

1195 · Flight Discount

Task · source analysis length 1,501 characters

Problem statement

Your task is to find a minimum-price flight route from Syrjälä to Metsälä. You have one discount coupon, using which you can halve the price of any single flight during the route. However, you can only use the coupon once.

When you use the discount coupon for a flight whose price is x, its price becomes $\lfloor x/2 \rfloor$ (it is rounded down to an integer).

Input

The first input line has two integers n and m: the number of cities and flight connections. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Metsälä.

After this there are m lines describing the flights. Each line has three integers a, b, and c: a flight begins at city a, ends at city b, and its price is c. Each flight is unidirectional.

You can assume that it is always possible to get from Syrjälä to Metsälä.

Output

Print one integer: the price of the cheapest route from Syrjälä to Metsälä.

Constraints

- $2 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le a, b \le n$
- $1 \le c \le 10^9$

We create a graph that consists of $2n$ nodes arranged in two layers.

Each city is represented by two nodes. Node i corresponds to city i before using the coupon, and node $n + i$ corresponds to city i after using the coupon.

For each flight, we create three edges. When there is a flight from city a to city b with weight x, we create the edges (a, b, x), $(a, n + b, \lfloor x/2 \rfloor)$ and $(n + a, n + b, x)$. The second edge allows us to use the coupon once.

In the above graph, the length of the shortest path from node 1 to node $2n$ represents the shortest route with the flight discount.

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;
using ll = long long;

const ll INF = 1e18;

int main() {
    int n, m;
    cin >> n >> m;

    vector<vector<pair<int, int>>> graph(2 * n + 1);

    for (int i = 1; i <= m; i++) {
        int a, b, c;
        cin >> a >> b >> c;
        graph[a].emplace_back(b, c);
        graph[n + a].emplace_back(n + b, c);
        graph[a].emplace_back(n + b, c / 2);
    }

    vector<ll> distance(2 * n + 1, INF);
    vector<bool> visited(2 * n + 1);

    priority_queue<pair<ll, int>> queue;
    queue.emplace(0, 1);
    distance[1] = 0;
```

```
    while (!queue.empty()) {
        int node = queue.top().second;
        queue.pop();

        if (visited[node]) continue;
        visited[node] = true;

        for (auto [next_node, weight] : graph[node]) {
            ll new_distance = distance[node] + weight;
            if (new_distance < distance[next_node]) {
                distance[next_node] = new_distance;
                queue.emplace(-new_distance, next_node);
            }
        }
    }

    cout << distance[2 * n] << "\n";
}
```

1136 · Counting Paths

Task · source analysis length 2,569 characters

Problem statement

You are given a tree consisting of n nodes, and m paths in the tree.

Your task is to calculate for each node the number of paths containing that node.

Input

The first input line contains integers n and m: the number of nodes and paths. The nodes are numbered $1, 2, \ldots, n$.

Then there are $n - 1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Finally, there are m lines describing the paths. Each line contains two integers a and b: there is a path between nodes a and b.

Output

Print n integers: for each node $1, 2, \ldots, n$, the number of paths containing that node.

Constraints

- $1 \le n, m \le 2 \cdot 10^5$
- $1 \le a, b \le n$

Let us root the tree arbitrarily. Notice that, in a rooted tree, any path between some nodes a and b can be split into two paths:

1. the part from a to $\text{lca}(a, b)$,
2. the part from b to $\text{lca}(a, b)$.

In this problem we will process these parts separately. Each node will have two values:

1. the number of paths starting at that node (going up),
2. the number of paths ending at this node.

We will use a dynamic programming approach. The paths through this node can be calculated by adding up the amount of paths that come from the children and the amount of paths starting at this node.

We have to make sure to remove the paths that end at this node from our counter. Note that each path is counted twice at its highest node (the LCA node). Fortunately, the number of doubly counted paths at any node is simply the number of paths ending at that node.

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 200000;
int jmp[N][20];
int depth[N];
vector<int> g[N];

void dfs(int node, int parent) {
    jmp[node][0] = parent;
    for (int child : g[node]) {
        if (child == parent) continue;
        depth[child] = depth[node] + 1;
        dfs(child, node);
    }
}

int lca(int a, int b) {
    if (depth[a] < depth[b]) {
        swap(a, b);
    }
    int depth_difference = depth[a] - depth[b];
    for (int j = 19; j >= 0; --j) {
        if ((1 << j) & depth_difference) {
            a = jmp[a][j];
        }
```

```
    }
    if (a == b) {
        return a;
    } else {
        for (int j = 19; j >= 0; --j) {
            if (jmp[a][j] != jmp[b][j]) {
                a = jmp[a][j];
                b = jmp[b][j];
            }
        }
        return jmp[a][0];
    }
}

int paths_starting[N];
int paths_ending[N];
int answer[N];

int dfs_paths(int node, int parent) {
    int current_paths = paths_starting[node];
    for (int child : g[node]) {
        if (child == parent) continue;
        current_paths += dfs_paths(child, node);
    }
    // Paths ending at the current node are counted twice.
    current_paths -= paths_ending[node];
    answer[node] = current_paths;
    current_paths -= paths_ending[node];
    return current_paths;
}

int main() {
    int n, m;

    cin >> n >> m;

    for (int i = 0; i < n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        a--;
        b--;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(0, 0);

    for (int j = 0; (1 << j) <= n; ++j) {
        for (int i = 0; i < n; ++i) {
            jmp[i][j + 1] = jmp[jmp[i][j]][j];
        }
    }

    for (int i = 0; i < m; ++i) {
        int a, b;
        cin >> a >> b;
        a--;
        b--;
        int c = lca(a, b);
        paths_starting[a]++;
        paths_starting[b]++;
        paths_ending[c]++;
    }

    dfs_paths(0, 0);

    for (int i = 0; i < n; ++i) {
        cout << answer[i] << ' ';
    }
    cout << '\n';
}
```

References

- CPHB (Competitive Programmer's Handbook), Chapter 18

MATH

3397 · Permutation Order

Task · source analysis length 1,763 characters

Problem statement

Let $p(n, k)$ denote the kth permutation (in lexicographical order) of $1 \ldots n$. For example, $p(4, 1) = [1, 2, 3, 4]$ and $p(4, 2) = [1, 2, 4, 3]$.

Your task is to process two types of tests:

1. Given n and k, find $p(n, k)$
2. Given n and $p(n, k)$, find k

Input

The first line has an integer t: the number of tests.

Each test is either "1 n k" or "2 n $p(n, k)$".

Output

For each test, print the answer according to the example.

Constraints

- $1 \le t \le 1000$
- $1 \le n \le 20$
- $1 \le k \le n!$

First task

We are given n and k, and our task is to find $p(n, k)$, i.e., the kth permutation (in lexicographical order) of $1 \ldots n$.

If k is between 1 and $(n-1)!$, the first number is 1. If k is between $(n-1)!+1$ and $2(n-1)!$, the first number is 2. In general, the first number is

$$\left\lfloor \frac{k-1}{(n-1)!} \right\rfloor + 1.$$

We can find the numbers one by one using the same idea. After every step, we remove the chosen number from the list of possible numbers.

Second task

We are given n and $p(n, k)$ and our task is to find k.

Initially $k = 1$. If the first number is 1, we don't change k. If the first number is 2, we increase k by $(n-1)!$. If the first number is 3, we increase k by $2(n-1)!$. In general, if the first number is x, we increase k by

$$(x-1)(n-1)!.$$

We can process the numbers one by one and increase the value of k using the same idea. After every step, we remove x from the list of numbers.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;
using ll = long long;
const int N = 20;

int main() {
    int t;
    cin >> t;

    vector<ll> fact(N + 1);
```

```
    fact[0] = 1;
    for (int i = 1; i <= N; i++) {
        fact[i] = fact[i - 1] * i;
    }

    while (t--) {
        int op;
        cin >> op;

        int n;
        cin >> n;

        ll f = fact[n];
        vector<int> numbers(n);
        iota(numbers.begin(), numbers.end(), 1);

        if (op == 1) {
            ll k;
            cin >> k;
            k--;
            for (int i = 0; i < n; i++) {
                f /= n - i;
                cout << numbers[k / f] << " ";
                numbers.erase(numbers.begin() + (k / f));
                k %= f;
            }
            cout << "\n";
        }

        if (op == 2) {
            ll k = 1;
            for (int i = 0; i < n; i++) {
                int x;
                cin >> x;
                f /= n - i;
                auto it = find(numbers.begin(), numbers.end(), x);
                k += (it - numbers.begin()) * f;
                numbers.erase(it);
            }
            cout << k << "\n";
        }
    }
}
```

3355 · Sum of Four Squares

Task · source analysis length 1,057 characters

Problem statement

A well known result in number theory is that every non-negative integer can be represented as the sum of four squares of non-negative integers.

You are given a non-negative integer n. Your task is to find four non-negative integers a, b, c and d such that $n = a^2 + b^2 + c^2 + d^2$.

Input

The first line has an integer t: the number of test cases.

Each of the next t lines has an integer n.

Output

For each test case, print four non-negative integers a, b, c and d that satisfy $n = a^2 + b^2 + c^2 + d^2$.

Constraints

- $1 \le t \le 1000$
- $0 \le n \le 10^7$
- the sum of all n is at most 10^7

Let $N = 10^7$ denote the maximum value of n.

We first find all integers between 0 and N that can be expressed as a sum of two squares. This can be done in $O(N)$ time. For each such integer $x = a^2 + b^2$, we store the first number a and the second number b.

After that, we can find a solution for any n in $O(n)$ time by finding a number x such that both x and $n - x$ can be expressed as a sum of two squares. It can be shown that there is always a solution.

```
#include <iostream>
#include <vector>
using namespace std;
const int N = 1e7;

int main() {
    int t;
    cin >> t;

    vector<bool> can(N + 1);
    vector<int> first(N + 1), second(N + 1);
    for (int a = 0; a * a <= N; a++) {
        for (int b = 0; a * a + b * b <= N; b++) {
            int x = a * a + b * b;
            can[x] = true;
            first[x] = a;
            second[x] = b;
        }
    }

    while (t--) {
        int n;
        cin >> n;
        for (int x = 0; x <= n; x++) {
            if (can[x] && can[n - x]) {
                cout << first[x] << " " << second[x] << " ";
                cout << first[n - x] << " " << second[n - x] << "\n";
                break;
            }
        }
    }
}
```

References

- Lagrange's four-square theorem (Wikipedia)

2214 · Inverse Inversions

Task · source analysis length 1,877 characters

Problem statement

Your task is to create a permutation of numbers $1, 2, \ldots, n$ that has exactly k inversions.

An inversion is a pair (a, b) where $a < b$ and $p_a > p_b$ where p_i denotes the number at position i in the permutation.

Input

The only input line has two integers n and k.

Output

Print a line that contains the permutation. You can print any valid solution.

Constraints

- $1 \le n \le 10^6$
- $0 \le k \le \frac{n(n-1)}{2}$

There is a straightforward greedy approach to this problem. However, it is not immediately clear that it works.

Let us construct the answer from the beginning, one element at a time. We will maintain two pointers: the lowest and highest numbers that have not been used, and at each point, append either of them to the answer and update the corresponding pointer.

Adding the lowest number yet will not form new inversions with any number following it as all of them will be larger. Adding the highest number yet, on the other hand, forms inversions with all of the numbers following it.

The greedy idea is to always pick the larger number while the inversion count remains under k. Now, we have to prove that the algorithm always finds a solution with exactly k inversions. In total, we have $n - 1$ choices: during the first step, we are offered $n - 1$ inversions, then $n - 2$ and so on until we are offered to take 1 inversion during the second-to-last step. The sum of the amounts is $1 + 2 + \cdots + (n - 1) = \frac{n(n-1)}{2}$, which means there are always enough inversions to form k.

It can be proven that because we take the inversions greedily, the amount of inversions needed is at all times at most $\frac{n(n-1)}{2}$, where n represents the number of elements remaining. It follows that when $n = 1$, the number of inversions needed is at most 0, thus equal to 0. The algorithm always finds a solution whose inversion count is never larger k nor is it lower than k.

```
#include <iostream>
using namespace std;
using ll = long long;

int main() {
    int n;
    ll k;
    cin >> n >> k;

    int l = 1, r = n;
    for (int i = 1; i <= n; ++i) {
        int left = n - i;  // The amount of numbers left.
        // As r is larger than all remaining numbers,
        // adding it will form inversions with each one of them.
        if (left <= k) {
            k -= left;
            cout << r-- << ' ';
        } else {
            cout << l++ << ' ';
        }
    }
    cout << '\n';
}
```

2210 · Counting Grids

Task · source analysis length 1,710 characters

Problem statement

Your task is to count the number of different $n \times n$ grids whose each square is black or white.

Two grids are considered to be different if it is not possible to rotate one of them so that they look the same.

Input

The only input line has an integer n: the size of the grid.

Output

Print one integer: the number of grids modulo $10^9 + 7$.

Constraints

- $1 \le n \le 10^9$

We solve the problem using Burnside's lemma which states that the number of distinct combinations is

$$\frac{1}{n} \sum_{i=1}^{n} c(i)$$

when there are n ways to change the position of a combination and there are $c(i)$ combinations that remain the same when the ith way is applied.

In this problem there are 4 ways to rotate the grid: $0°$, $90°$, $180°$, and $270°$. Our task is to count the combinations that remain the same after each rotation. In each case we count the number of squares whose color we can choose.

When n is even, the numbers are as follows:

- $0°$: n^2
- $90°$: $\frac{n^2}{4}$
- $180°$: $\frac{n^2}{2}$
- $270°$: $\frac{n^2}{4}$

When n is odd, the numbers are as follows:

- $0°$: n^2
- $90°$: $\lfloor \frac{n}{2} \rfloor \lfloor \frac{n}{2} + 1 \rfloor + 1$
- $180°$: $n \lfloor \frac{n}{2} \rfloor + \lfloor \frac{n}{2} + 1 \rfloor$
- $270°$: $\lfloor \frac{n}{2} \rfloor \lfloor \frac{n}{2} + 1 \rfloor + 1$

```
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;

int power(int a, ll b) {
    if (b == 0) return 1;
    ll p = power(a, b / 2);
    p = p * p % M;
    if (b % 2 == 1) p = p * a % M;
    return p;
}

int inv(int x) {
    return power(x, M - 2);
}

int main() {
    ll n;
    cin >> n;

    ll sum = 0;
    if (n % 2 == 0) {
```

```
        sum += power(2, n * n);
        sum += power(2, n * n / 4);
        sum += power(2, n * n / 2);
        sum += power(2, n * n / 4);
    } else {
        sum += power(2, n * n);
        sum += power(2, (n / 2) * (n / 2 + 1) + 1);
        sum += power(2, n * (n / 2) + (n / 2 + 1));
        sum += power(2, (n / 2) * (n / 2 + 1) + 1);
    }
    ll answer = sum * inv(4) % M;
    cout << answer << "\n";
}
```

References

- Burnside's lemma (Wikipedia)

2209 · Counting Necklaces

Task · source analysis length 1,138 characters

Problem statement

Your task is to count the number of different necklaces that consist of n pearls and each pearl has m possible colors.

Two necklaces are considered to be different if it is not possible to rotate one of them so that they look the same.

Input

The only input line has two numbers n and m: the number of pearls and colors.

Output

Print one integer: the number of different necklaces modulo $10^9 + 7$.

Constraints

- $1 \le n, m \le 10^6$

We solve the problem using Burnside's lemma which states that the number of distinct combinations is

$$\frac{1}{n} \sum_{i=1}^{n} c(i)$$

when there are n ways to change the position of a combination and there are $c(i)$ combinations that remain the same when the ith way is applied.

In this problem there are n ways to rotate the necklace: we can rotate it $0, 1, \ldots, n-1$ steps. The number of combinations that remain the same is

$$m^{\gcd(i,n)}$$

because there is a repeating block of $\gcd(i, n)$ pearls and there are m ways to choose the color of each pearl inside the block.

```
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int power(int a, int b) {
    if (b == 0) return 1;
    ll p = power(a, b / 2);
    p = p * p % M;
    if (b % 2 == 1) p = p * a % M;
    return p;
}

int inv(int x) {
    return power(x, M - 2);
}

int main() {
    int n, m;
    cin >> n >> m;

    ll sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += power(m, gcd(i, n));
        sum %= M;
    }
    ll answer = sum * inv(n) % M;
```

```
    cout << answer << "\n";
}
```

References

- Burnside's lemma (Wikipedia)

2182 · Divisor Analysis

Task · source analysis length 4,783 characters

Problem statement

Given an integer, your task is to find the number, sum and product of its divisors. As an example, let us consider the number 12:

- the number of divisors is 6 (they are 1, 2, 3, 4, 6, 12)
- the sum of divisors is $1+2+3+4+6+12=28$
- the product of divisors is $1 \cdot 2 \cdot 3 \cdot 4 \cdot 6 \cdot 12 = 1728$

Since the input number may be large, it is given as a prime factorization.

Input

The first line has an integer n: the number of parts in the prime factorization.

After this, there are n lines that describe the factorization. Each line has two numbers x and k where x is a prime and k is its power.

Output

Print three integers modulo 10^9+7: the number, sum and product of the divisors.

Constraints

- $1 \le n \le 10^5$
- $2 \le x \le 10^6$
- each x is a distinct prime
- $1 \le k \le 10^9$

We are given a number

$$N = \prod_{i=1}^{n} x_i^{k_i}$$

and our task is to calculate the number, sum and product of its divisors.

Solution 1

We calculate the number of divisors as follows:

$$d(N) = \prod_{i=1}^{n} (k_i+1)$$

The formula works because there are k_i+1 ways to choose a factor x_i.

We calculate the sum of divisors as follows:

$$s(N) = \prod_{i=1}^{n} (1 + x_i + x_i^2 + \cdots + x_i^{k_i}) = \prod_{i=1}^{n} \frac{x_i^{k_i+1}-1}{x_i-1}$$

This formula corresponds to a sum where each divisor appears exactly once. Since each part of the product is a geometric sum, we can calculate the answer more efficiently using the geometric sum formula.

We calculate the product of divisors as follows:

$$p(N) = \prod_{i=1}^{n} (x_i^{k_i(k_i+1)/2})^{d(N)/(k_i+1)}$$

This formula works because if a factor x_i appears in a divisor, it can appear between 1 and k_i times, so we can use the sum formula $k_i(k_i+1)/2$ to count the total number of factor appearances. Moreover, there are

$d(N)/(k_i+1)$ ways to choose the other factors.

Note that since we want to calculate the answer modulo M, we have to calculate $d(N)/(k_i+1)$ modulo $M-1$. We can't use modular multiplicative inverses here because k_i+1 and $M-1$ are not necessarily coprime. Instead, we create two additional vectors that store prefix and suffix products of k_i+1 values and use them in the last loop.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

ll power(ll a, ll b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

ll inv(ll x) {
    return power(x, M - 2);
}

int main() {
    int n;
    cin >> n;

    vector<ll> x(n), k(n);
    for (int i = 0; i < n; i++) {
        cin >> x[i] >> k[i];
    }

    ll count = 1;
    for (int i = 0; i < n; i++) {
        count = count * (k[i] + 1) % M;
    }

    ll sum = 1;
    for (int i = 0; i < n; i++) {
        sum = sum * (power(x[i], k[i] + 1) - 1) % M;
        sum = sum * inv(x[i] - 1) % M;
    }

    vector<ll> prefix(n + 1);
    prefix[0] = 1;
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] * (k[i] + 1) % (M - 1);
    }

    vector<ll> suffix(n);
    suffix[n - 1] = 1;
    for (int i = n - 1; i > 0; i--) {
        suffix[i - 1] = suffix[i] * (k[i] + 1) % (M - 1);
    }

    ll prod = 1;
    for (int i = 0; i < n; i++) {
        ll ways = prefix[i] * suffix[i] % (M - 1);
        prod *= power(power(x[i], k[i] * (k[i] + 1) / 2), ways);
        prod %= M;
    }

    cout << count << " " << sum << " " << prod << "\n";
}
```

Solution 2

A more elegant way to calculate the product of divisors is as follows:

$$p(N) = N^{d(N)/2}$$

We can derive this formula from the previous formula by simplifying the exponents:

$$p(N) = \prod_{i=1}^{n} (x_i^{k_i(k_i+1)/2})^{d(N)/(k_i+1)} = \prod_{i=1}^{n} (x_i^{k_i})^{d(N)/2} = N^{d(N)/2}$$

The following solution calculates the product of divisors using this formula and does not require additional vectors.

We need to calculate $d(N)/2$ modulo $M-1$ and we can't use modular multiplicative inverses because $M-1$ and 2 are not coprime. We handle two cases in the code:

Case 1: N is not square. There is an even value k_i+1 and we divide the first such value by two to calculate the exponent $d(N)/2$.

Case 2: N is square. Now $d(N)$ is odd and $d(N)/2$ is not an integer. However, $\sqrt{N}$ is an integer so we can use the formula $N^{d(N)/2} = \sqrt{N}^{d(N)}$ to calculate the answer. In this case all k_i values are even and we divide each of them by two in the last loop.

Another way to explain the formula $N^{d(N)/2}$ is to consider divisor pairs of the form $(k, N/k)$ where $k < N/k$. When N is not square, there are $d(N)/2$ pairs and the product of each pair is N. When N is square, there are $(d(N)-1)/2$ pairs with product N and then an additional divisor $\sqrt{N} = N^{1/2}$. The formula holds also in this case because

$$N^{(d(N)-1)/2} N^{1/2} = N^{d(N)/2}.$$

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

ll power(ll a, ll b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

ll inv(ll x) {
    return power(x, M - 2);
}

int main() {
    int n;
    cin >> n;

    vector<ll> x(n), k(n);
    for (int i = 0; i < n; i++) {
        cin >> x[i] >> k[i];
    }

    ll count = 1;
    for (int i = 0; i < n; i++) {
        count = count * (k[i] + 1) % M;
    }

    ll sum = 1;
    for (int i = 0; i < n; i++) {
        sum = sum * (power(x[i], k[i] + 1) - 1) % M;
        sum = sum * inv(x[i] - 1) % M;
    }

    ll count2 = 1;
    bool is_square = true;
    for (int i = 0; i < n; i++) {
        if (is_square && k[i] % 2 == 1) {
            count2 *= (k[i] + 1) / 2;
            is_square = false;
        } else {
            count2 *= (k[i] + 1);
        }
        count2 %= M - 1;
    }

    ll prod = 1;
    for (int i = 0; i < n; i++) {
        if (is_square) k[i] /= 2;
        prod *= power(power(x[i], k[i]), count2);
        prod %= M;
    }
```

```
    cout << count << " " << sum << " " << prod << "\n";
}
```

2164 · Josephus Queries

Task · source analysis length 1,788 characters

Problem statement

Consider a game where there are n children (numbered $1, 2, \ldots, n$) in a circle. During the game, every second child is removed from the circle, until there are no children left.

Your task is to process q queries of the form: "when there are n children, who is the kth child that will be removed?"

Input

The first input line has an integer q: the number of queries.

After this, there are q lines that describe the queries. Each line has two integers n and k: the number of children and the position of the child.

Output

Print q integers: the answer for each query.

Constraints

- $1 \le q \le 10^5$
- $1 \le k \le n \le 10^9$

We can divide the process of removing children into rounds where we go through the children in increasing order and remove every second of them. For example, when $n = 7$, the rounds are as follows:

- Round 1: $[1, \underline{2}, 3, \underline{4}, 5, \underline{6}, 7]$
- Round 2: $[1, 3, \underline{5}, 7]$
- Round 3: $[\underline{3}, 7]$
- Round 4: $[\underline{7}]$

To efficiently find the kth removed child, we simulate the rounds using a recursive function whose each call corresponds to a round.

Let $\mathrm{child}(n, k, p)$ denote the kth child who will be removed when there are n children and the parity of the first child to be removed is p (0 means we first remove child 2, and 1 means we first remove child 1).

The number of children removed on the first round is $r = \lfloor n + p \rfloor / 2$. If $k \le r$, we remove the kth child on the first round and just return the number of the child which is $2k - p$.

Otherwise we call the function recursively: we have $n - r$ children left and we should find the $(k - r)$th child who will be removed. Here u denotes the new parity: if n is even, the parity remains the same, and if n is odd, the parity changes. Since the numbering of the children changes in the subproblem, we have to convert it after the recursive call.

Since we remove half of the children on each round, the number of rounds is $O(\log n)$ and the function works in $O(\log n)$ time.

```
#include <iostream>
using namespace std;

int child(int n, int k, int p) {
    int r = (n + p) / 2;
    if (k <= r) return k * 2 - p;
    int u = n % 2 == 0 ? p : 1 - p;
    return child(n - r, k - r, u) * 2 - 1 + p;
}

int main() {
    int q;
    cin >> q;

    for (int qi = 1; qi <= q; qi++) {
        int n, k;
        cin >> n >> k;
        cout << child(n, k, 0) << "\n";
```

```
    }
}
```

References

- Josephus problem (Wikipedia)

2137 · Corner Subgrid Count

Task · source analysis length 2,437 characters

Problem statement

You are given an $n \times n$ grid whose each square is either black or white. A subgrid is called *beautiful* if its height and width is at least two and all of its corners are black. How many beautiful subgrids are there within the given grid?

Input

The first input line has an integer n: the size of the grid.

Then there are n lines describing the grid: `1` means that a square is black and `0` means it is white.

Output

Print the number of beautiful subgrids.

Constraints

- $1 \le n \le 3000$

Going through all possible corners of a subgrid would run in $O(n^4)$ and thus, be too slow. There is, however, a way to calculate the number of subgrids by only going through each pair of rows.

Let us go through each pair of rows and find each subgrid which spans from one of the selected rows to another. For a pair of rows to form a subgrid, the rows need to share two distinct positions which have black squares in both of the rows. If we know the number of such positions is c, the number of subgrids the rows form, is $\binom{c}{2} = \frac{c(c-1)}{2}$, as we may choose any two of them as the borders of the subgrid. This algorithm runs in $O(n^3)$ as we go through each pair of rows in $O(n)$ time and there are in total $O(n^2)$ pairs of rows.

With $n = 3000$, the solution would perform roughly $3000^3 \approx 3 \cdot 10^{10}$ operations and would still be too slow. We may, however, use bitsets to improve the constant factor of the solution. Bitsets allow us to check multiple positions at once. On a Linux system, bitsets usually contain 64-bit integers. This means that, with constant time bit operations, we can check 64 positions at once. Thus, the number of operations becomes $3000^3 \cdot \frac{1}{64} \approx 4 \cdot 10^8$.

To find the positions where the two rows both have a black square, we will use the bitwise AND operation as it discards the positions where only one or neither of the squares are black. Finally, we will count the number of bits set in the bitset using the method `.count()`. Note that even with the bitset optimization the solution will turn out too slow without the line

```
#pragma GCC target("popcnt")
```

which tells the compiler that our machine has an instruction for calculating the popcount of an integer, that is, our processor is able to count the number of bits set in an integer efficiently. The popcount instruction is available on CSES judges and has been included in most of Intel's x86-64 processors since 2008.

```
#pragma GCC target("popcnt")
#include <bitset>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
using B = bitset<3000>;

int main() {
    int n;
    cin >> n;

    vector<B> b(n);
    for (int i = 0; i < n; ++i) {
        string s;
        cin >> s;
        for (int j = 0; j < n; ++j) {
            if (s[j] == '1') {
                b[i][j] = true;
            }
```

```
        }
    }

    ll r = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            int c = (b[i] & b[j]).count();
            r += c * (c - 1) / 2;
        }
    }

    cout << r << '\n';
}
```

1728 · Inversion Probability

Task · source analysis length 1,784 characters

Problem statement

An array has n integers $x_1, x_2, \ldots, x_n$, and each of them has been randomly chosen between 1 and r_i. An inversion is a pair (a, b) where $a < b$ and $x_a > x_b$.

What is the expected number of inversions in the array?

Input

The first input line contains an integer n: the size of the array.

The second line contains n integers $r_1, r_2, \ldots, r_n$: the range of possible values for each array position.

Output

Print the expected number of inversions rounded to six decimal places (rounding half to even).

Constraints

- $1 \le n \le 100$
- $1 \le r_i \le 100$

We consider each index pair (a, b) where $a < b$ and calculate the probability that $x_a > x_b$ which corresponds to an inversion.

Using linearity of expectation, the sum of the probabilities equals the expected number of inversions.

C++ solution

Here is a C++ solution for the problem:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> r(n);
    for (int i = 0; i < n; i++) {
        cin >> r[i];
    }

    double answer = 0;
    for (int a = 0; a < n; a++) {
        for (int b = a + 1; b < n; b++) {
            for (int x_a = 1; x_a <= r[a]; x_a++) {
                for (int x_b = 1; x_b <= min(x_a - 1, r[b]); x_b++) {
                    answer += 1.0 / (r[a] * r[b]);
                }
            }
        }
    }
    cout.precision(6);
    cout << fixed << answer << "\n";
}
```

Unfortunately, there are test cases where this solution fails due to precision issues. In general, it is difficult to create a correct solution in C++.

Python solution

In Python, we can use the `Fraction` class to avoid precision issues:

```
from fractions import Fraction

n = int(input())
r = [int(x) for x in input().split(" ")]
```

```
answer = Fraction(0, 1)
for a in range(n):
    for b in range(a + 1, n):
        for x_a in range(1, r[a] + 1):
            for x_b in range(1, min(x_a, r[b] + 1)):
                answer += Fraction(1, r[a] * r[b])

answer = float(round(answer, 6))
print(f"{answer:06f}")
```

The above solution works correctly but it is too slow. However, we can optimize the solution as follows:

```
from fractions import Fraction

n = int(input())
r = [int(x) for x in input().split(" ")]

answer = Fraction(0, 1)
for a in range(n):
    for b in range(a + 1, n):
        count = 0
        for x_a in range(1, r[a] + 1):
            count += min(x_a - 1, r[b])
        answer += Fraction(count, r[a] * r[b])

answer = float(round(answer, 6))
print(f"{answer:06f}")
```

This solution works both correctly and efficiently enough.

1726 · Moving Robots

Task · source analysis length 2,067 characters

Problem statement

Each square of an 8×8 chessboard has a robot. Each robot independently moves k steps, and there can be many robots on the same square.

On each turn, a robot moves one step left, right, up or down, but not outside the board. It randomly chooses a direction among those where it can move.

Your task is to calculate the expected number of *empty* squares after k turns.

Input

The only input line has an integer k.

Output

Print the expected number of empty squares rounded to six decimal places (rounding half to even).

Constraints

- $1 \le k \le 100$

We assume that the rows and columns are numbered from 0 to $N-1$ and the robots are numbered from 0 to N^2-1. The robot that starts at square (i,j) has number $i \cdot N + j$. In this problem $N = 8$ in each test.

We solve the problem using dynamic programming: $p[m][r][i][j]$ denotes the probability that robot r is in square (i,j) after m turns.

As base cases, $p[0][i \cdot N + j][i][j] = 1$ for each square (i,j). Then, when we know the value of $p[m][r][i][j]$, we find the squares where robot r can move from square (i,j). For each such square (i',j'), we increase the value of $p[m+1][r][i'][j']$ by $p[m][r][i][j]/c$ where c is the total number of squares where the robot can move.

After that, we go through the grid and calculate, for each square (i,j), the probability that the square is empty after k turns. We calculate the probability using the formula

$$\prod_{r=0}^{N^2-1} (1 - p[k][r][i][j])$$

which requires that no robot is in the square after k turns. Due to linearity of expectation, the sum of all such probabilities equals the expected number of empty squares after k turns.

The solution works in $O(kN^4)$ time.

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 8;
const int K = 100;
double p[K + 1][N * N][N][N];

int main() {
    int k;
    cin >> k;

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            p[0][i * N + j][i][j] = 1;
        }
    }

    for (int m = 0; m < k; m++) {
        for (int r = 0; r < N * N; r++) {
            for (int i = 0; i < N; i++) {
                for (int j = 0; j < N; j++) {
                    vector<pair<int, int>> moves;
                    if (i > 0) moves.emplace_back(i - 1, j);
                    if (j > 0) moves.emplace_back(i, j - 1);
                    if (i + 1 < N) moves.emplace_back(i + 1, j);
```

```
                    if (j + 1 < N) moves.emplace_back(i, j + 1);
                    double add = p[m][r][i][j] / moves.size();
                    for (auto d : moves) {
                        p[m + 1][r][d.first][d.second] += add;
                    }
                }
            }
        }
    }

    double answer = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double e = 1;
            for (int r = 0; r < N * N; r++) {
                e *= (1 - p[k][r][i][j]);
            }
            answer += e;
        }
    }
    cout.precision(6);
    cout << fixed << answer << "\n";
}
```

1725 · Dice Probability

Task · source analysis length 923 characters

Problem statement

You throw a dice n times, and every throw produces an outcome between 1 and 6. What is the probability that the sum of outcomes is between a and b?

Input

The only input line contains three integers n, a and b.

Output

Print the probability rounded to six decimal places (rounding half to even).

Constraints

- $1 \le n \le 100$
- $1 \le a \le b \le 6n$

We solve the problem using dynamic programming: $p[i][j]$ denotes the probability of having sum j after throwing a dice i times.

As a base case $p[0][0] = 1$, because the sum is certainly 0 if there are no throws. Then, when we know a value of $p[i][j]$, we go through the possible throws whose outcomes are $k = 1, 2, \ldots, 6$. For each such throw, we increase $p[i+1][j+k]$ by $p[i][j]/6$. Finally, we calculate the sum of probabilities $p[n][i]$ where i is between a and b.

The solution works in $O(n^2)$ time.

```
#include <iostream>
using namespace std;

const int N = 100;
double p[N + 1][6 * N + 1];

int main() {
    int n, a, b;
    cin >> n >> a >> b;

    p[0][0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = i; j <= 6 * i; j++) {
            for (int k = 1; k <= 6; k++) {
                p[i + 1][j + k] += p[i][j] / 6;
            }
        }
    }

    double answer = 0;
    for (int i = a; i <= b; i++) {
        answer += p[n][i];
    }
    cout.precision(6);
    cout << fixed << answer << "\n";
}
```

1082 · Sum of Divisors

Task · source analysis length 1,924 characters

Problem statement

Let $\sigma(n)$ denote the sum of divisors of an integer n. For example, $\sigma(12) = 1+2+3+4+6+12 = 28$.
Your task is to calculate the sum $\sum_{i=1}^{n} \sigma(i)$ modulo 10^9+7.

Input

The only input line has an integer n.

Output

Print $\sum_{i=1}^{n} \sigma(i)$ modulo 10^9+7.

Constraints

- $1 \le n \le 10^{12}$

Solution 1

Since a divisor k appears in $\lfloor n/k \rfloor$ numbers between 1 and n, the problem can be restated as calculating the sum

$$\sum_{k=1}^{n} \lfloor n/k \rfloor k.$$

The following solution calculates the sum in $O(n)$ time. It is a good start but it is too slow when $n = 10^{12}$.

```
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;

int main() {
    ll n;
    cin >> n;

    ll s = 0;
    for (ll k = 1; k <= n; k++) {
        s += (n / k) * k;
        s %= M;
    }
    cout << s << "\n";
}
```

Solution 2

We can improve the first solution by calculating the sum in two parts.

First, we use the original formula for small divisors between 1 and $\lfloor \sqrt{n} \rfloor$. Then, we calculate the rest of the sum by considering blocks of large divisors where each divisor appears the same number of times.

For example, we know that the divisors between $\lfloor n/2 \rfloor + 1$ and n appear once, the divisors between $\lfloor n/3 \rfloor + 1$ and $\lfloor n/2 \rfloor$ appear twice, and so on. More generally, the divisors between $\lfloor n/(k+1) \rfloor + 1$ and $\lfloor n/k \rfloor$ appear k times.

We calculate the sum of divisors in a block using the sum formula

$$a + (a+1) + \cdots + b = \frac{(a+b)(b-a+1)}{2}.$$

Note that since the sum may be large, we use the multiplicative inverse of 2 modulo M to divide the product by 2.

We calculate both parts of the sum in $O(\sqrt{n})$ time, so the algorithm works in $O(\sqrt{n})$ time which is enough when $n = 10^{12}$.

```cpp
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;
const int INV2 = 500000004;

// a + (a+1) + ... + b
ll sum(ll a, ll b) {
    return (a + b) % M * ((b - a + 1) % M) % M * INV2 % M;
}

int main() {
    ll n;
    cin >> n;

    ll s = 0;
    for (ll k = 1; k * k <= n; k++) {
        s += (n / k) * k;
        s %= M;
    }
    for (ll k = 1; n / k > k; k++) {
        s += sum(n / (k + 1) + 1, n / k) * k;
        s %= M;
    }
    cout << s << "\n";
}
```

2229 · Permutation Inversions

Task · source analysis length 1,992 characters

Problem statement

Your task is to count the number of permutations of $1, 2, \ldots, n$ that have exactly k inversions (i.e., pairs of elements in the wrong order).

For example, when $n = 4$ and $k = 3$, there are 6 such permutations:

- $[1, 4, 3, 2]$
- $[2, 3, 4, 1]$
- $[2, 4, 1, 3]$
- $[3, 1, 4, 2]$
- $[3, 2, 1, 4]$
- $[4, 1, 2, 3]$

Input

The only input line has two integers n and k.

Output

Print the answer modulo $10^9 + 7$.

Constraints

- $1 \le n \le 500$
- $0 \le k \le \frac{n(n-1)}{2}$

Solution 1: $O(n^4)$ approach

We will first present a $O(n^4)$ approach. The approach is a bit too slow, but it can be easily sped up. Let us use dynamic programming to compute the answer. Let $\mathrm{dp}[i][j]$ represent the number of permutations of the numbers $1 \ldots i$ with exactly j inversions.

Now, when we are placing the number $i + 1$, we have $i + 1$ places for it. If we place the number at the end of the permutation, no new inversion will be formed. Each place to the left of this will add a new inversion as all numbers are smaller than the new one.

These observations lead to a solution where we loop through the numbers, the place for each number and the number of previous inversions. This leads to the recurrence

$$\mathrm{dp}[i+1][j] = \sum_{0 \le \mathrm{pos} \le i} \mathrm{dp}[i][j - \mathrm{pos}].$$

The table may be filled with this recurrence in $O(n^4)$ time.

```
#include <iostream>
using namespace std;

const int M = 1e9 + 7;
const int N = 501;
const int K = 124751;
int dp[N][K];

int main() {
    int n, k;
    cin >> n >> k;

    dp[0][0] = 1;
    for (int i = 1; i <= n; ++i) {
        for (int h = 0; h <= k; ++h) {
            for (int j = 0; j <= min(h, i - 1); ++j) {
                dp[i][h] += dp[i - 1][h - j];
                dp[i][h] %= M;
            }
```

```
        }
    }

    cout << dp[n][k] << '\n';
}
```

Solution 2: $O(n^3)$ approach

Note that the value of each dp position corresponds to a sum of a range of values from the previous layer. Further, the ranges are all the same length and their starting positions differ by one. We can, thus, maintain the sum while filling the dp layer. This leads to a solution with a time complexity of $O(n^3)$.

```
#include <iostream>
using namespace std;

const int M = 1e9 + 7;
const int N = 501;
const int K = 124751;
int dp[N][K];

int main() {
    int n, k;
    cin >> n >> k;

    dp[0][0] = 1;
    for (int i = 1; i <= n; ++i) {
        int sum = 0;
        for (int h = 0; h <= k; ++h) {
            sum += dp[i - 1][h];
            sum %= M;
            if (h >= i) {
                sum -= dp[i - 1][h - i];
                sum += M;
                sum %= M;
            }
            dp[i][h] = sum;
        }
    }

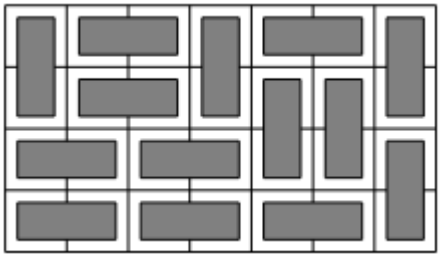
    cout << dp[n][k] << '\n';
}
```

2181 · Counting Tilings

Task · source analysis length 4,023 characters

> **Problem statement**
>
> Your task is to count the number of ways you can fill an $n \times m$ grid using 1×2 and 2×1 tiles.
>
> **Input**
>
> The only input line has two integers n and m.
>
> **Output**
>
> Print one integer: the number of ways modulo $10^9 + 7$.
>
> **Constraints**
>
> - $1 \le n \le 10$
> - $1 \le m \le 1000$

We use dynamic programming to solve the problem. We represent each column pattern as a bitmask that indicates the rows where a horizontal tile begins. For example, consider the following tiling:



Here the bitmasks 0011, 1100, 0011, 0000, 1001, 0000 and 0000 correspond to the column patterns from left to right.

A pair of bitmasks (a, b) is valid if patterns a and b can occur next to each other in the tiling. For example, $(0011, 1100)$ is a valid pair which corresponds to the first two columns in the example. The pair $(0011, 1101)$ is not valid, because a horizontal tile begins in the last row in both bitmasks.

A simple solution would be to iterate over all pairs of bitmasks in every column. There are 4^n bitmask pairs and checking if a pair is valid takes $O(n)$ time, so such a solution would require $O(mn4^n)$ time. This can be optimized by only checking pairs of bitmasks which share no bits. There are exactly 3^n such bitmasks, which would result in an $O(mn3^n)$ solution.

An $O(mn3^n)$ solution can be made fast enough, but since in practice there are much less valid transitions than 3^n, and the same transitions are used for each column, it makes sense to precompute them.

Solution 1: Precomputed transitions

In this solution, the transitions are computed recursively before performing dynamic programming, one or two bits at a time. The bitmask `left` represents which rows have a horizontal tile going left, and `right` represents which rows have a horizontal tile going right. Other rows are occupied by vertical tiles, which have a height of two.

Dynamic programming values from only the previous column need to be known, so two vectors are used and swapped between columns.

It can be shown [2] that the number of transitions is $O((1 + \sqrt{2})^n)$. Therefore the time complexity of this solution is $O(m(1 + \sqrt{2})^n)$, or approximately $O(m2.42^n)$.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
const int M = 1000000007;

int n;
```

```
vector<pair<int, int>> transitions;

void generate(int i, int left, int right) {
    if (i > n) return;
    if (i == n) {
        transitions.emplace_back(left, right);
        return;
    }
    generate(i + 1, left | 1 << i, right);
    generate(i + 1, left, right | 1 << i);
    generate(i + 2, left, right);
}

int main() {
    int m;
    cin >> n >> m;

    generate(0, 0, 0);

    vector<int> cur(1 << n), prev(1 << n);
    prev[0] = 1;

    for (int i = 0; i < m; ++i) {
        fill(cur.begin(), cur.end(), 0);

        for (auto [x, y] : transitions) {
            cur[y] += prev[x];
            cur[y] %= M;
        }

        swap(cur, prev);
    }

    cout << prev[0] << endl;
}
```

Solution 2: Sum over subsets

Let us take a closer look at the transitions.

One transition that is always valid is from the bitmask s to the complement of s. At every position where there is not a tile going to the left, we choose a tile going to the right. Complementing the bitmasks is equivalent to reversing the dynamic programming array.

The other transitions from s are formed by replacing some adjacent pairs of tiles going to the right with vertical pieces.

The sum over subsets technique can be used to compute the desired sum of valid transitions. More precisely, as the value associated with the bitmask s we sum all values of supersets of s that may have some added adjacent pairs of 1-bits. The technique works similarly even with this added restriction.

The time complexity of this solution is $O(mn2^n)$. For $n = 10$ it is very similar in performance to the first solution.

```
#include <algorithm>
#include <iostream>
using namespace std;
const int M = 1000000007;

int tilings[1 << 10];

int main() {
    int n, m;
    cin >> n >> m;

    tilings[0] = 1;

    for (int i = 0; i < m; ++i) {
        reverse(tilings, tilings + (1 << n));

        for (int j = 0; j < n - 1; ++j) {
            for (int s = 0; s < (1 << n); ++s) {
                if ((s & 0b11 << j) == 0) {
                    tilings[s] += tilings[s | 0b11 << j];
                    tilings[s] %= M;
                }
            }
        }
    }
```

```
    cout << tilings[0] << endl;
}
```

References

1. CPHB (Competitive Programmer's Handbook), Chapters 7.6 and 10.6, Counting subsets.
2. Pell number (Wikipedia)

1653 · Elevator Rides

Task · source analysis length 3,731 characters

Problem statement

There are n people who want to get to the top of a building which has only one elevator. You know the weight of each person and the maximum allowed weight in the elevator. What is the minimum number of elevator rides?

Input

The first input line has two integers n and x: the number of people and the maximum allowed weight in the elevator.

The second line has n integers $w_1, w_2, \ldots, w_n$: the weight of each person.

Output

Print one integer: the minimum number of rides.

Constraints

- $1 \le n \le 20$
- $1 \le x \le 10^9$
- $1 \le w_i \le x$

Faulty greedy approaches

Initially it might seem that a greedy approach could solve the problem: choose people into one elevator ride as long as someone fits. However, no simple way of choosing the next person is known that would work optimally in all cases.

For example, consider the input

$$x = 8$$
$$w = 4, 3, 3, 2, 2, 2.$$

Neither a strategy which always chooses the heaviest person that still fits in the elevator, nor one that chooses the lightest, are able to find the optimal solution, which is two full elevators:

$$(4, 2, 2), (3, 3, 2)$$

Permutations

Iterating through all permutations of people suffices to solve the problem. The idea is to add people to the elevator one by one as long as the next person fits, and starting a new ride otherwise. Any elevator assignment will correspond to some permutation from which the solution can be reconstructed.

There are at most $20! \approx 10^{18}$ different permutations, so each one of them cannot be checked individually. The idea can be optimized by instead doing dynamic programming over subsets of people, also called bitmask dynamic programming.

Permutations to subsets

When choosing people one by one, the only information that needs to be considered is the *subset* of people that have already been assigned elevator rides. The order in which the previous people were chosen does not matter as long as it is optimal.

Let the function best have values of the form $\mathrm{best}(S) = (r, l)$ where the pair (r, l) describes the optimal number of elevator rides r and weight already used from the last ride l for any subset S of people. Among solutions with the same amount of rides, the one with the least l is considered better.

The function can be computed recursively by individually trying each person in the subset as the last person to be considered.

$$\text{best}(S) = \min_{p \in S} \text{add}(\text{best}(S \setminus \{p\}), w_p)$$

Here, $\text{add}((r, l), w_p)$ tries to add the person p into the last elevator ride as $(r, l + w_p)$, or establishes a new one as $(r + 1, w_p)$ if $l + w_p > x$.

We set $\text{best}(\emptyset) = (1, 0)$ as the base case to make the rest of the values correct even though zero rides are technically needed.

It would be possible to make the amount of elevator rides a variable of the function, as $\text{best}(S, r) = l$, and minimize only the total weight of the last elevator ride. However, this is not necessary because there is never any ambiguity comparing pairs (r, l). That is, a pair with less rides is always strictly better than one with more. This allows us to find the optimal pair much like a single number can be optimized with dynamic programming. In fact, the default comparison operators for `std::pair` work in the correct way.

The subset S is conveniently represented as a bitmask in the solution below. Iterating over the bitmasks in the order corresponding to their numeric values guarantees that all smaller subsets are computed before the current one. The solution has a time complexity of $O(n2^n)$.

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 20;
pair<int, int> best[1 << N];

int main() {
    int n, x;
    cin >> n >> x;

    vector<int> w(n);
    for (int i = 0; i < n; ++i) cin >> w[i];

    best[0] = {1, 0};
    for (int s = 1; s < (1 << n); ++s) {
        best[s] = {n + 1, 0};

        for (int p = 0; p < n; ++p) {
            if (s & (1 << p)) {
                auto [r, l] = best[s ^ (1 << p)];
                pair<int, int> added;
                if (l + w[p] <= x) {
                    added = {r, l + w[p]};
                } else {
                    added = {r + 1, w[p]};
                }
                best[s] = min(best[s], added);
            }
        }
    }

    cout << best[(1 << n) - 1].first << endl;
}
```

References

- CPHB (Competitive Programmer's Handbook), Chapter 10, From permutations to subsets